

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

6. Hadoop - MapReduce	1
6.1. Distribuciones de Hadoop	1
6.1.1. Cloudera	1
6.1.2. MapR	2
6.1.3. HortonWorks	2
6.1.4. Amazon EMR - Elastic <i>map reduce</i>	3
6.1.5. Microsoft Azure	4
6.1.6. Comparativa entre distribuciones	5
6.1.7. Instalación de Cloudera	5
6.2. Componentes de Hadoop	6
6.2.1. Modos de Funcionamiento	7
6.2.2. Componentes de Hadoop	8
6.3. Manejo de HDFS	10
6.4. MapReduce	14
6.4.1. Hadoop MapReduce	16

Listado de acrónimos

EMR	Elastic Map Reduce
HDFS	Hadoop Distributed File System
RAM	Random Access Memory

6

Capítulo

Hadoop - MapReduce

Francisco P. Romero

6.1. Distribuciones de Hadoop

En el ecosistema Hadoop existen diversas distribuciones que implementan una arquitectura común a diversas necesidades de Big Data. Estas integran un conjunto de herramientas de ingesta de datos, control de flujo, almacenamiento, análisis y procesamiento. Entre estas distribuciones, que pueden ser implementadas en la infraestructura interna de la organización, las más usadas en los proyectos de Big Data.[EN16] son Cloudera, MapR y HortonWorks. Además, existen soluciones desplegadas de servicios en la nube (cloud) que, en muchos casos, pueden coincidir con implementaciones de dichas distribuciones. Las dos plataformas más conocidas en este ámbito son: Amazon EMR (Elastic MapReduce) y Microsoft Azure.

6.1.1. Cloudera

Cloudera es una de las distribuciones de Hadoop más conocidas y usadas. Este agrega un conjunto de componentes propietarios al ecosistema Hadoop; estos componentes fueron diseñados para optimizar la gestión de los clústeres y brindar mejores experiencias de búsquedas. Alguno de los componentes desarrollados por Cloudera son:

- **Impala:** es un motor basado en SQL, a tiempo real y paralelizado, realiza búsquedas de datos en el sistema de archivos HDFS.
- **Cloudera Manager:** es la consola que permite gestionar y desplegar los componentes en el clúster Hadoop.

- **Hue:** es una consola que le permite al usuario interactuar con los datos y ejecutar scripts para los diferentes componentes de Hadoop contenidos en el clúster.

La Figura 6.1 ilustra la distribución Hadoop de Cloudera, sus componentes se clasifican de la siguiente manera:

- Componentes que son parte del núcleo de Hadoop: YARN y HDFS.
- Componentes que son parte del ecosistema Hadoop: Zookeeper, Hbase, Hive, Pig, Flume, Oozie y Sqoop.
- Componentes desarrollados por Cloudera: Cloudera Manager, Impala y Hue.

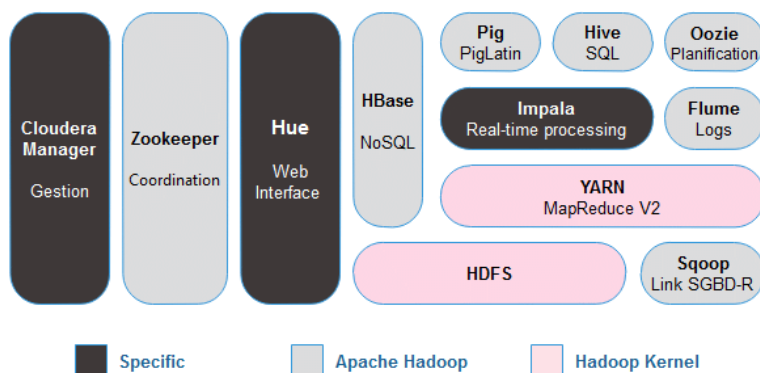


Figura 6.1: Distribucion Cloudera. Fuente:Cloudera Inc.

6.1.2. MapR

Actualmente distribuida como HPE, era una distribución que tenía como pieza clave al sistema de archivos MapR. Implementa la API de HDFS, es de lectura/escritura completa y puede almacenar miles de millones de archivos (en comparación con la configuración compleja para HDFS que requiere espacios de nombres separados). Los usuarios de esta plataforma normalmente tienen o están planeando clústeres Hadoop de misión crítica para lo cual pueden usar MapR-DB y MapR Streams (que implementan las APIs HBase y Kafka, respectivamente)[EBT17].

6.1.3. HortonWorks

Hortonworks ofrece una distribución cien por ciento de código abierto. Integra componentes estables del proyecto Hadoop antes que las últimas versiones lanzadas. Agrega *Ambari* como un componente de gestión en su arquitectura, el que es comparable con Cloudera Manager.

Hortonworks posee las siguientes características:

- Implementación: en la infraestructura interna, en la nube o en una arquitectura híbrida; ya sea sobre plataformas Linux o Windows.
- Se recolectan y almacenan los datos en su forma original, sin importar su fuente o formato.
- Permite recoger datos de las bases de datos relacionales existentes, además de tipos de datos menos estructurados como el flujo de interacción o clics en sitios web (web clickstream), datos de máquinas y sensores, social media, datos desde móviles, geolocalización o los datos de registros de servidores.
- Tiene a YARN como su núcleo principal.

En la Figura 6.2 se puede apreciar los principales componentes que forman parte de esta distribución

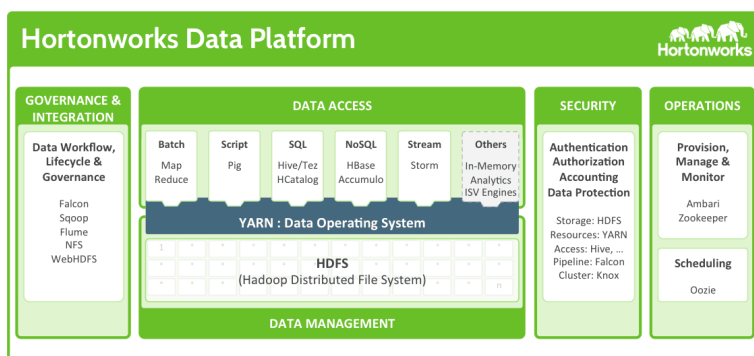


Figura 6.2: Distribucion HortonWorks. Fuente: HortonWorks

6.1.4. Amazon EMR - Elastic *map reduce*

Esta plataforma permite analizar y procesar grandes cantidades de datos mediante la distribución del trabajo de cómputo, a través de un clúster de servidores virtuales. El clúster es gestionado bajo la plataforma Hadoop, donde un nodo es designado como maestro para controlar la distribución de tareas.

El servicio *Amazon EMR* ha realizado mejoras y personalizaciones en los componentes de Hadoop para trabajar de forma integrada con *Amazon Web Services* (AWS). Los clústeres Hadoop que se ejecutan en *Amazon EMR* usan las siguientes instancias: *Amazon Elastic Compute Cloud* (Amazon EC2) que consiste en servidores virtuales Linux para los nodos maestro y esclavo, *Amazon Simple Storage Service* (Amazon S3) para el consumo y almacenamiento masivo de datos y *Cloud-Watch* para supervisar el rendimiento del clúster y generación de alertas.

6.1.5. Microsoft Azure

La plataforma *Azure* implementa una solución denominada *HDInsight* que utiliza la distribución *Hortonworks Data Platform* (HDP) basada en Hadoop[Sar14]. Esta solución permite implementar y gestionar clústeres de Apache Hadoop en la nube con el fin de proporcionar un sistema óptimo para procesar, analizar y generar informes garantizando una alta confiabilidad y disponibilidad. Además proporciona la posibilidad del aprovisionamiento automático de clústeres de Hadoop.

Sus características principales son las siguientes:

- Constante actualización de los componentes Hadoop.
- Alta disponibilidad y confiabilidad de los clústeres.
- Almacenamiento de datos eficiente y económico con el almacenamiento de blobs de Azure, opción compatible con Hadoop.
- Integración con otros servicios de Azure, como aplicaciones web y bases de datos SQL.
- Escalado de clústeres, que permite cambiar el número de nodos de un clúster *HDInsight* en tiempo de ejecución.
- Compatibilidad con redes virtuales, lo que permite el aislamiento de los recursos en la nube o en escenarios híbridos para poder vincular los recursos de la nube con recursos locales.

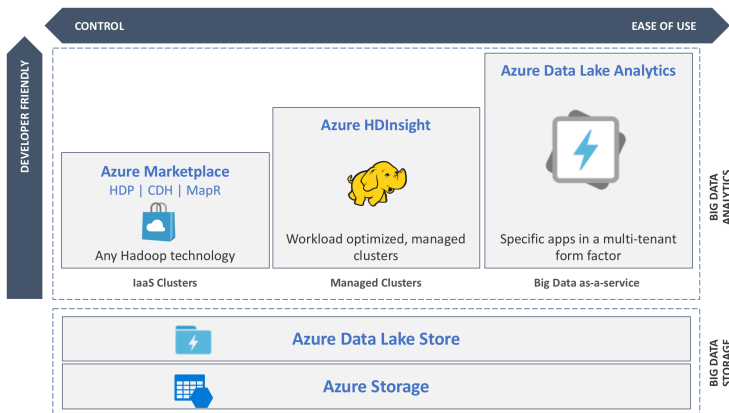


Figura 6.3: Elementos principales de Microsoft Azure HDInsight. Fuente: Microsoft Azure

6.1.6. Comparativa entre distribuciones

Para poder determinar que distribución implementar dentro de un proyecto Big Data, es necesario evaluar las características del problema, así como, sus necesidades. Una vez realizado el análisis del proyecto a implementar y definida la arquitectura a utilizar, se deben evaluar las opciones de implementación disponibles que asemejen o implementen las herramientas necesarias para alcanzar el objetivo del proyecto. Se debe considerar que al implementar un proyecto Big Data se ha de utilizar diversas fuentes de datos y, según sea el caso, crear diferentes almacenes de datos estructurados y no estructurados.

Las distribuciones como Cloudera y Hortonworks son equivalentes en la construcción de una arquitectura y tienen un alto grado de madurez lo que las convierten en plataformas líderes del mercado. *Cloudera* al ser una distribución con mayor madurez, ofrece herramientas personalizadas que sobresalen por su funcionalidad y eficiencia frente a las demás distribuciones. *Hortonworks* nos permite una adaptación más rápida a las nuevas versiones del ecosistema Hadoop.

Por último, las soluciones en la nube como Amazon EMR y HDInsight de Microsoft Azure nos brindan la posibilidad de implementar una solución Big Data externalizada o híbrida por lo que se tiene que asumir costos por el uso de los nodos de un clúster en un tiempo determinado. Se debe considerar que al hacer uso de estos servicios se genera una alta dependencia con el proveedor. Del mismo modo, se debe tener muy en cuenta las políticas y condiciones en el tratamiento de los datos de dichas plataformas.

6.1.7. Instalación de Cloudera

La instalación y las pruebas de este curso se han realizado sobre una instalación de la distribución Cloudera ejecutada como máquina virtual sobre Virtual Box.

Los requisitos para esta ejecución son: utilizar un Sistema Operativo de 64 bit, instalar la última versión de Virtual Box con el fin de poder manejar el fichero distribuible de la máquina virtual. La memoria RAM recomendable es de al menos 4GB aunque para poder ejecutar con solvencia las prácticas se recomiendan al menos 8GB.

La máquina virtual Cloudera en sus últimas versiones únicamente está disponible sobre registro en Cloudera, de todas formas se cuenta con distribuciones accesibles que pueden ser bajadas de Internet (cerca de 6GB). En este caso utilizaremos la versión 5.13 🐉 Enlace: <http://bit.ly/3DIYCPv>

Una vez que se lanza la máquina virtual el usuario se encuentra automáticamente logeado como usuario cloudera (usuario: cloudera, password: cloudera). Esta cuenta tiene privilegios de superusuario en la máquina virtual y la contraseña de root es la misma (cloudera).

6.2. Componentes de Hadoop

Como se ha mencionado anteriormente Hadoop es un *framework* que permite almacenar y procesar conjuntos de datos de gran tamaño de forma paralela y distribuida. Los dos componentes más importantes de este *framework* son:

- **Hadoop Distributed File System:** Se ocupa de las tareas de almacenamiento y replicación de los datos masivos. Los principales componentes de este sistema son:
 - NameNode (NN) Nodo de Nombres.
 - Secondary NameNode (SNN): Nodo de Nombres Secundario.
 - DataNode (DN): Nodos de Datos
- **MapReduce:** Permite el procesamiento paralelo de los datos almacenados en HDFS. Los principales componentes de procesamiento de esta parte son los siguientes:
 - Job Tracker (JT)
 - Task Tracker (TT)

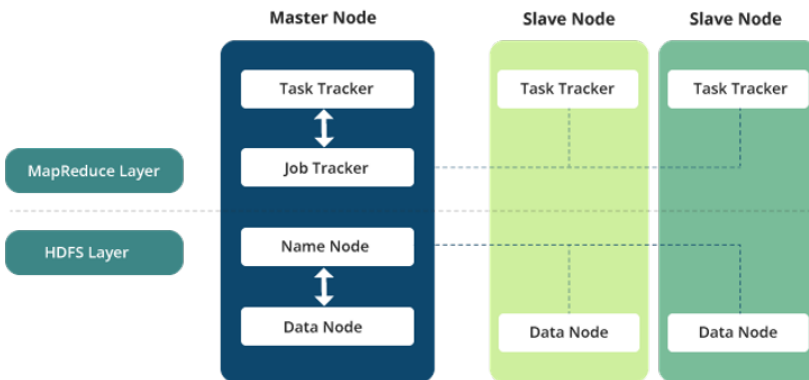


Figura 6.4: Estructura de Hadoop. Fuente: www.besanttechnologies.com

6.2.1. Modos de Funcionamiento

Existen dos modos de funcionamiento en Hadoop: standalone, pseudo-distribuido.

Modo Standalone

El modo **standalone** viene definido por las siguientes características:

- El modo *standalone* es el modo por defecto cuando se ejecuta Hadoop.
- Se utiliza principalmente para depuración cuando realmente no se está utilizando HDFS de forma distribuida como sistema de almacenamiento de soporte.
- El sistema de archivos local se puede utilizar como entrada y/o salida de los diferentes procesos que se prueben.
- No es necesario configurar los diferentes componentes propios de Hadoop dado que se funciona en un modo local. Por lo tanto no se modificarán archivos como `mapred-site.xml`, `core-site.xml` o `HDFS-site.xml`.
- Es el modo más rápido de ejecución de Hadoop ya que utiliza el sistema de archivos local como entrada y salida lo cual *reduce* el coste con comunicaciones, etc.

Modo Pseudo-distribuido

El modo pseudo-distribuido corresponde a la ejecución de Hadoop con la configuración de un solo nodo (*single-node cluster*) en la que tanto Nodo de Nombres y Nodos de datos están localizados en la misma máquina física.

En este modo, todos los procesos internos de Hadoop (daemons) se van a ejecutar en un único nodo. Esta configuración se utiliza principalmente en la fase de pruebas en la cual, el uso de los recursos distribuidos no es un factor crítico y no es necesario contar con la compartición de recursos a otros usuarios.

Los componentes que forman parte de Hadoop se comunicaran a través de la red mediante *sockets*, lo cual permite contar con un clúster mínimo de un solo nodo completamente funcional y optimizado. Además, se genera una máquina virtual Java independiente para cada Hadoop.

En cuanto a HDFS, el factor de replicación de los bloques en este caso será de uno y no de un número superior al no contar con más nodos.

Por último, si que será necesario realizar diferentes cambios en los ficheros de configuración: `mapred-site.xml`, `core-site.xml` o `HDFS-site.xml`.



Un socket es un punto final de un enlace de comunicación bidireccional entre dos programas que se ejecutan en la red. Un socket está vinculado a un número de puerto para que la capa TCP pueda identificar la aplicación a la que están destinados los datos.

Modo Completamente Distribuido

Es el modo de producción de Hadoop en la que varios nodos están en ejecución desempeñando sus diferentes roles en la arquitectura Hadoop. Los datos están distribuidos en diferentes nodos y el procesamiento se ejecutará de forma distribuida en cada nodo. De esta forma los servicios en modo Maestro y Esclavo (*Master and Slave*) se ejecutarán en nodos separados.

Este modo de funcionamiento es el modo de funcionamiento real de Hadoop, aunque los dos modos anteriores tienen su utilidad en las fases de prueba y depuración.

6.2.2. Componentes de Hadoop

NameNode (NN)

Es un componente clave en la arquitectura Hadoop ya que es el que gobierna el funcionamiento del Cluster. El NameNode puede ejecutarse sobre cualquier infraestructura hardware ya que en sí es un componente software que hace que el componente actúe de servidor maestro ejecutando tareas como las siguientes:

- Gestionar el espacio de nombres del sistema de archivos distribuido. Esto lo hace mediante:
 - **fsimage:** Este archivo tiene la información completa sobre los metadatos del sistema de archivos cuando se inicia el NameNode. Todas las operaciones posteriores se registran en *editlog*.
 - **editlog:** Todas las operaciones de escritura de archivos realizadas por las aplicaciones cliente se registran primero en el este fichero..

Sobre cada archivo se almacenan como metadatos su nombre, su ruta, el número de bloques, los identificadores (*ids*) de estos bloques y el nivel de replicación.

- Gestionar el acceso a los archivos de los diferentes clientes y usuarios, es decir, es el responsable del acceso seguro y mediante privilegios establecidos a los datos del sistema.
- Recibe los bloques de datos del resto de nodos, así como otros mecanismos de verificación de funcionamiento (*heartbeats*).

- Manejar los ficheros mediante operaciones de renombrado, cierre, apertura, etc. tanto de ficheros como de directorios.

DataNode (DN)

Por cada nodo en el cluster va a haber un nodo de Datos o DataNode. Estos nodos gestionarán el almacenamiento de todo el sistema, ejecutando, de acuerdo a las peticiones de los procesos cliente, las operaciones de lectura y escritura de ficheros en los sistemas de archivos de cada nodo.

Aunque esta arquitectura soporta diferentes sistemas de archivos generalmente se utilizará HDFS como base del sistema de archivos. De esta manera, un archivo dentro de un sistema de archivos va a dividirse en uno o más segmentos que se almacenarán de forma individual en los nodos de datos. Estos segmentos, denominados bloques son la mínima cantidad de datos que HDFS puede leer, escribir o replicar. El tamaño de estos bloques es de 64MB por defecto, muy orientados a manejar grandes ficheros, pero puede verse incrementado utilizando la configuración correspondiente en HDFS.

De esta manera, el DataNode se ocupa de operaciones como la creación, borrado y replicación de bloques siguiendo las instrucciones del nodo de Nombres (NameNode).

Secondary Name Node (SNN)

Uno de los posibles puntos débiles de la arquitectura Hadoop es la disponibilidad del nodo de Nombres (NameNode). En el caso de que éste falle todo el sistema de archivos distribuido deja de funcionar. Con el fin de solventar este problema existe el nodo de nombres secundario cuya principal función es almacenar una copia de los ficheros *fsimage* y *editlog*.

De esta forma, su función principal es comprobar los metadatos del sistema de archivos almacenados en NameNode. Esto es lo que se llama *checkpointing*.

El proceso que sigue el NameNode secundario para fusionar periódicamente los archivos *fsimage* y *edits log* es el siguiente:

1. El NameNode secundario obtiene los últimos archivos *fsImage* y *editLog* del NameNode primario.
2. El NameNode secundario aplica cada transacción del archivo *editLog* a *fsImage* para crear un nuevo archivo *FsImage* fusionado.
3. El archivo *fsImage* fusionado se transfiere de nuevo al NameNode primario.

JobTracker y TaskTracker

Un trabajo (*job*) es dividido en múltiples tareas que se ejecutarán en los múltiples DataNodes que existen en el cluster. Para cada trabajo enviado al sistema para su ejecución existirá un JobTracker que residirá en el NameNode, y habrá varios TaskTrackers que se ejecutarán en los DataNode.

El JobTracker se ocupará de coordinar la actividad mediante la planificación de las tareas que se deben ejecutar en los diferentes nodos de datos. Estas tareas estarán controladas por el TaskTracker correspondiente de cada uno de los nodos de datos. Estos procesos se ocuparán de informar al JobTracker del progreso en la realización de las tareas, esto lo podrá hacer mediante el mecanismo que se denomina *heartbeat*, señal que se enviará al JobTracker para informar del estado del nodo de datos.

El JobTracker mantendrá el control del progreso de cada uno de los trabajos distribuidos, de esta manera, en caso de error en la ejecución de las tareas, éste puede replanificarla a otro TaskTracker y en consecuencia a otro nodo de datos.

6.3. Manejo de HDFS

HDFS es el sistema de ficheros primero que Hadoop utiliza para ejecutar trabajos MapReduce. Su manejo es similar al de un sistema de archivos como el de Linux, permitiendo un manejo mediante línea de comandos muy eficiente.

Antes de ejecutar cualquier comando Hadoop es recomendable verificar si todos los procesos *demonio* están activos mediante el siguiente comando. Si no lo están será necesario chequear el estado de HDFS y reiniciar si se requiere.

```
$ sudo jps
```

El comando para hacer que el NameNode abandone el modo seguro. El NameNode durante este modo recopila toda la información de metadatos desde el NameNode secundario.

```
$ sudo -u hdfs dfs dfsadmin -safemode leave
```



El safemode es como la ejecución en modo seguro de Windows y sirve para restaurar la información después de una caída. A veces al subir ficheros con *Hue* a la máquina virtual Cloudera arroja un error de NameNode en nodos seguro, este es el comando que hay que ejecutar.

Todos los comandos HDFS pueden comenzar de dos maneras distintas o bien `hadoop fs`, lo cual es soportado por cualquier tipo de sistema de archivos, o bien `hdfs fs` que es equivalente al anterior pero exclusivo para HDFS.



En la distribución de cloudera, el sistema de archivos local por defecto está localizado en `/home/cloudera` y la localización por defecto de HDFS es `/user/cloudera`

- **Crear un directorio:** Para crear un nuevo directorio dentro del sistema de ficheros HDFS.

```
$ hadoop fs -mkdir /user/cloudera/prueba
```



Atención a los permisos necesarios para crear directorios en diferentes puntos del sistema de archivos

- **Listar contenidos de un directorio:** Para ver los contenidos del directorio HDFS el comando es el siguiente:

```
$ hadoop fs -ls /user/cloudera
```

- **Copiar un fichero** del sistema de archivos local al sistema de archivos HDFS. Se podrá verificar esa copia mediante el comando `hadoop fs -ls` o mediante la operación `cat` que se menciona a continuación.

```
$ hadoop fs -copyFromLocal /home/cloudera/fichero /user/cloudera
```



Por defecto, el destino de cualquier operación HJDFS es `/user/cloudera`, de manera que es opcional especificar esa parte de la ruta salvo que sea diferente a la de por defecto.

- **Visualización del contenido de un archivo:** Para ver el contenido de un archivo ubicado en el sistema de archivo HDFS la operación será la siguiente::

```
$ hadoop fs -cat /user/cloudera/prueba/fichero
```

- **Extraer un fichero** del sistema de archivos HDFS: Con el fin de copiar a nuestros sistema de archivos local un archivo del sistema de archivos de HDFS se utilizará alguno de los siguientes comandos.

```
$ hadoop fs -copyToLocal /user/cloudera/prueba/fichero  
$ hadoop fs -get /user/cloudera/prueba/fichero
```


- **Mover ficheros dentro de HDFS:** Para mover ficheros almacenados en HDFS a otros directorios de HDFS se podría utilizar el siguiente comando:

```
$ hadoop fs -mv /user/cloudera/prueba/fichero /user/cloudera/ $
```

Se permiten múltiples orígenes de ficheros, lo que obliga a que el destino sea un directorio. El movimiento de ficheros entre diferentes sistemas de archivos no está permitido.

- **Copiar ficheros dentro de HDFS:** Copia un fichero entre diferentes localizaciones dentro de HDFS. También, como `mv` permite copiar desde diferentes orígenes, pero siempre con un directorio de destino final.

```
$ hadoop fs -cp -f /user/cloudera/prueba/fichero /user/cloudera/
```



la opción `-f` permitirá sobrescribir el destino si éste existe previamente.

- **Put:** Permite copiar uno o varios orígenes desde el sistema de archivos local al sistema de archivos distribuido.

```
$ hadoop fs -put /user/cloudera/prueba/fichero /user/cloudera/
```

También permite leer desde la entrada estándar (`stdin`) y escribe en el sistema de archivos destino.

```
$ hadoop fs -put - /user/cloudera/entrada
```

- **Añadir contenido al final del fichero:** A veces es necesario hacer operaciones de concatenación de ficheros, etc., para ello existe la operación `appendToFile` que permite hacer esta operación.

```
$ hadoop fs -appendToFile fichero_tail /user/cloudera/fichero
```

- **Mezcla de ficheros:** se utiliza para combinar varios archivos (o directorios) del sistema de archivos distribuido y luego ponerlo en un solo archivo de salida en nuestro sistema de archivos local. Dispone de una opción `-nl` con el fin de añadir una nueva línea al final de cada fichero.

```
$ hdfs dfs -getmerge -nl file1.txt file2.txt /home/cloudera/output.txt
```

- **Borrado de ficheros:** La operación `rm` permitirá borrar los ficheros especificados como argumentos. Con la opción `-R` se borrarán el directorio y los subdirectorios de forma recursiva.

```
$ hadoop fs -rm -r /user/cloudera/prueba
```



La opción `-skipTrash` permitirá saltarse el paso de mover los ficheros a la papelera de reciclaje y de esta maneja borrarlos de forma inmediata

- **Cambio de permisos a los ficheros:** De la misma forma que en Linux la operación `chmod` permitirá realizar cambios en los permisos de uso de los ficheros. Con la opción `-R` hace que el cambio se propague recursivamente a través de la estructura de directorios.

```
$ hadoop fs -chmod -R 777 /user/cloudera/prueba
```



Si el sistema deniega el permiso de ejecutar esta operación ejecutar como superusuario

- **Comprobar uso de disco:** Servirá para comprobar cuando espacio de disco se está usando en HDFS. Si estamos interesados únicamente en el uso de disco de nuestro directorio de usuario el comando será:

```
$ hadoop fs -du
```

Si por el contrario queremos conocer cuando espacio de disco está disponible en todo el cluster, el comando será:

```
$ hadoop fs -df
```

- **Contar número de directorios:** El siguiente comando permite obtener la información del número de directorios , ficheros y tamaño de los mismos.

```
$ hadoop fs -count /user/cloudera
```

- **touchz:** Crea un fichero vacío.

```
$ hadoop fs -touchz /user/cloudera/emptyfile
```

6.4. MapReduce

El modelo de programación MapReduce fue introducido por primera vez por Google a finales del año 2004. El objetivo era crear un framework adaptado a la computación paralela que permitiera procesar y generar grandes colecciones de datos sobre máquinas genéricas, sin la necesidad de utilizar supercomputadores o servidores dedicados, y que fuera fácilmente escalable.

En esencia, el modelo que propone MapReduce es bastante sencillo. El programador debe encargarse de implementar dos funciones, *map* y *reduce*, que serán aplicadas a todos los datos de entrada. Tareas como hacer particiones de los datos de entrada, despliegue de maestro y trabajadores, esquema de asignación de trabajos a los trabajadores, comunicación y sincronización entre procesos, tratamiento de caídas de procesos, quedan a cargo del entorno de ejecución, liberando de esa manera al programador.

Las funciones *map* y *reduce* están definidas ambas con respecto a datos estructurados en parejas <clave, valor>. La función *map* es aplicada en paralelo sobre cada pareja de entrada $(k1, v1)$, generando una lista intermedia de parejas <clave, valor>($lista(k2, v2)$). La función *reduce*, es aplicada en paralelo sobre cada grupo de parejas con la misma clave intermedia ($k2$), tomando los valores ($v2$) contenidos en el grupo para generar como salida una nueva lista de parejas <clave, valor>($lista(k3, v3)$). Entre las fases *map* y *reduce*, existe una operación de agrupación que reúne todos los pares con la misma clave de todas las listas, creando un grupo por cada una de las diferentes claves generadas. Las tuplas generadas por la función *reduce*, son grabadas de forma permanente en el sistema de ficheros distribuido. Por tanto, la ejecución de la aplicación acaba produciendo r ficheros, donde r es el número de tareas *reduce* ejecutadas. La Figura 6.7 muestra el esquema completo de ejecución de MapReduce.

Una de las ideas más importantes que hay detrás de MapReduce, es separar el qué se distribuye del cómo. Un trabajo *MapReduce* está formado por el código asociado a las funciones *map* y *reduce*, junto con parámetros de configuración. El desarrollador envía el trabajo al planificador y el entorno de ejecución se encarga de todo lo demás, gestionando el resto de aspectos relacionados con la ejecución distribuida del código, como son:

- **Planificación:** Cada aplicación MapReduce es dividida en unidades más pequeñas denominadas tareas (tasks). Dado que el número de tareas puede ser superior al de los nodos del clúster, es necesario que el planificador mantenga una cola de tareas y vaya haciendo un seguimiento del progreso de las tareas en ejecución, asignando tareas de la cola de espera a medida que los nodos van quedando disponibles.
- **Co-ubicación datos/código:** Una de las ideas clave de MapReduce es mover el código y no los datos. Esta idea está ligada con la planificación y depende, en gran medida, del diseño del sistema de ficheros distribuido. Para lograr la localidad de datos, el planificador intentará ejecutar la tarea en el nodo que contiene un determinado bloque de datos necesario para la tarea.

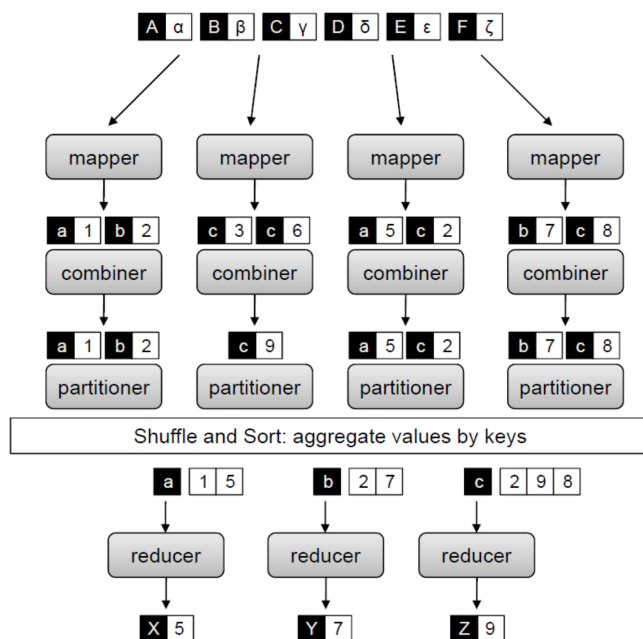


Figura 6.5: Esquema completo de ejecución MapReduce.

- **Sincronización:** En MapReduce, la sincronización se consigue mediante una “barrera”. Una “barrera”, es un mecanismo de sincronización entre procesos que espera a que todos los procesos de un lado de la barrera terminen antes que empiecen los procesos del otro lado. Esto significa que la fase *map* debe terminar antes que empiece la fase *reduce*.
- **Manejo de errores y fallos:** El entorno de ejecución debe cumplir con todas las tareas mencionadas en los puntos anteriores, en un entorno donde los errores y los fallos son la norma. MapReduce fue diseñado expresamente para ser ejecutados en servidores de gama baja, por lo que el entorno de ejecución debe ser especialmente resistente. En grandes clústeres de máquinas, los errores de disco y de RAM son comunes.

Existen dos elementos adicionales que completan el modelo de programación, denominados *combiner* y *partitioner*.

El *partitioner* es el responsable de dividir el espacio de claves intermedias y asignar parejas <clave, valor> intermedias a las tareas *reduce*. Dicho de otro modo, el *partitioner* especifica la tarea a la cual debe asignarse una pareja <clave, valor>. Para ello, calcula el valor *hash* de la clave (la transforma en un valor alfanumérico) y obtiene el módulo del valor en base al número de tareas *reduce* a ejecutar. El

objetivo es asignar el mismo número de claves a cada *reduce*. Sin embargo, dado que el *partitioner* sólo tiene en cuenta la clave y no los valores, puede haber grandes diferencias en el volumen de tuplas <clave, valor> enviadas a cada *reduce*, dado que diferentes claves pueden tener diferente número de valores asociados.

El *combiner* representa un punto de optimización en el modelo MapReduce. En algunos casos, existe una repetición significativa en las claves intermedias producidas por cada *map*. Todas esas repeticiones deberían ser enviadas, a través de la red, al nodo asignado para procesar una tarea *reduce* para una clave dada. Este modelo se muestra del todo ineficiente, por lo que el desarrollador tiene la opción de definir una función *combiner*. Esta función le permite fusionar, parcialmente, los datos antes de ser enviados a través de la red. La función *combiner* se ejecuta en cada nodo donde se procesa una tarea *map*. Típicamente, se utiliza el mismo código para implementar la función *combiner* y la función *reduce*. La diferencia radica en el modo en que MapReduce gestiona la salida de la función. La salida de la función *reduce* se escribe en el sistema de ficheros de forma permanente. La salida de la función *combiner* se escribe en un fichero intermedio que será enviado a una tarea *reduce*.

6.4.1. Hadoop MapReduce

La idea principal sobre la cual gira el entorno de ejecución Hadoop MapReduce es que la entrada puede ser dividida en fragmentos y, cada fragmento, puede ser tratado de forma independiente por una tarea *map*. Los resultados de procesar cada fragmento, pueden ser físicamente divididos en grupos distintos. Cada grupo se ordena y se pasa a una tarea *reduce*. Una tarea *map* puede ejecutarse en cualquier nodo de cómputo del clúster, y múltiples tareas *map* pueden ejecutarse en paralelo en el clúster. La tarea *map* es responsable de transformar los registros de entrada en parejas <clave, valor>. La salida de todos los *map* se dividirá en particiones, y cada partición será ordenada por clave. Habrá una partición por cada tarea *reduce*. Tanto la clave como los valores asociados a la clave son procesados por la tarea *reduce*, siendo posible que varias tareas *reduce* se ejecuten en paralelo.

El desarrollador únicamente proporciona al framework Hadoop cuatro funciones: la función que lee los registros de entrada y los transforma en tuplas <clave, valor> (RecordReader), la función *map* (Mapper), la función *reduce* (Reducer), y la función que transforma las parejas <clave, valor> generadas por la función *reduce* en registros de salida (RecordWriter).

El entorno de ejecución Hadoop Mapreduce, como ya se ha comentado, está formado por dos componentes principales: JobTracker y TaskTracker, ambos codificados en Java. Al igual que en el HDFS, el entorno de ejecución presenta una arquitectura cliente/servidor o master/slave (maestro/esclavo). En un clúster hay un único JobTracker, siendo su labor principal la gestión de los TaskTrackers, entre los que distribuye los trabajos MapReduce que recibe. Los TaskTrackers son los encargados de ejecutar las tareas *map/reduce*. En un clúster típico, se ejecuta un TaskTracker por nodo de cómputo. En la Figura 6.6 se muestra de forma esquemática la interacción entre JobTracker y TaskTracker.

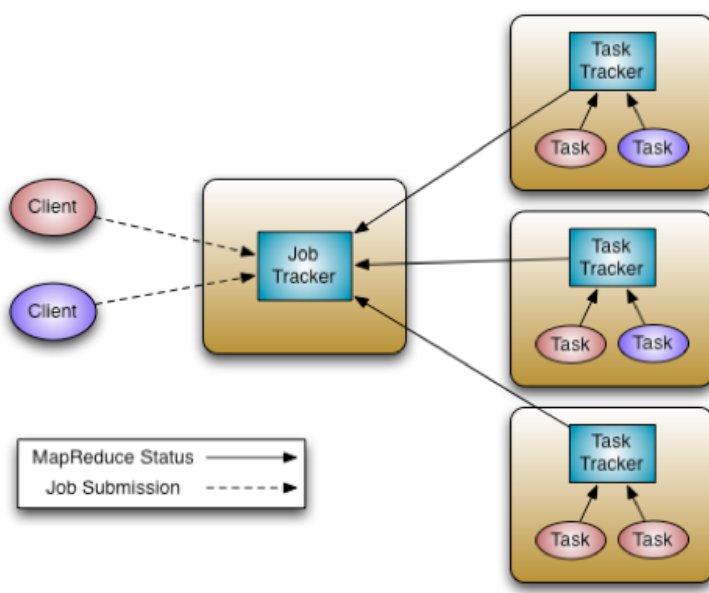


Figura 6.6: Workflow de ejecución MapReduce

El JobTracker es el enlace entre la aplicación y Hadoop. Una vez se envía un trabajo al clúster, el JobTracker determina el plan de ejecución en base a los ficheros a procesar, asigna nodos de cómputo a las diferentes tareas, y supervisa todas las tareas que se están ejecutando. En el caso de fallar una tarea, el JobTracker relanza la tarea, posiblemente en un nodo diferente, existiendo un límite predefinido de intentos en el caso que dicha tarea falle de forma reiterada. Como se ha comentado anteriormente, sólo se ejecuta un JobTracker por clúster Hadoop. Cada TaskTracker es responsable de ejecutar las tareas que el JobTracker le ha asignado. Cada TaskTracker puede ejecutar varias tareas *map/reduce* en paralelo. El JobTracker crea instancias Java separadas para la ejecución de cada tarea. De esa manera se garantiza que, en caso de fallar la ejecución de una tarea, no afecta al resto de tareas ni tampoco al propio JobTracker.

El TaskTracker se comunica con el JobTracker a través de un protocolo de *latidos* (*heart beat protocol*). En esencia, el *latido* es un mecanismo que utiliza el TaskTracker para anunciar su disponibilidad en el clúster. Este protocolo permite saber al JobTracker que el TaskTracker está disponible o en funcionamiento. Además de anunciar su disponibilidad, el protocolo de *latidos* también incluye información sobre el estado del TaskTracker. Estos mensajes indican si el TaskTracker está listo para la siguiente tarea o no. Cuando el JobTracker recibe un mensaje de latido de un TaskTracker, declarando que está listo para la siguiente tarea, el JobTracker selecciona el siguiente trabajo disponible en la lista de prioridades y determine qué tarea es la más apropiada para el TaskTracker.

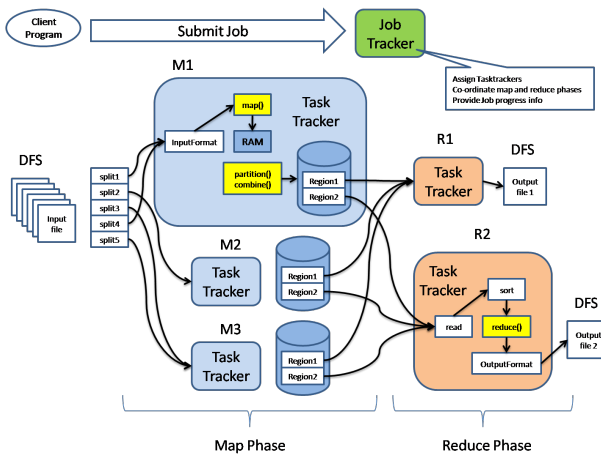


Figura 6.7: Esquema de Ejecución MapReduce

A continuación, se describe cada una de las fases que presenta cada proceso. Ya que la mayoría de fases son funciones internas del propio framework de ejecución, tienen asociados una serie de parámetros de configuración que pueden afectar al rendimiento del propio clúster. La configuración de dichos parámetros puede ser realizada por el propio programador o por el administrador del sistema.

Tarea *map*

- **Procesamiento *map***: a cada tarea *map* (*map* 1, *map* 2, ..., *map* n) se le asigna un bloque del fichero (Input Split). Cada bloque es dividido en parejas del tipo <clave, valor>. La función *map* es invocada para cada pareja. La información generada por la función *map* es escrita en un buffer de memoria circular asociado con cada tarea *map* (In Memory Buffer). El tamaño del buffer viene fijado por la propiedad *io.sort.mb*, siendo el tamaño por defecto de 100Mb.
- **Spill**: cuando el buffer llega al porcentaje de ocupación fijado por la propiedad *io.sort.spill.percent* (por defecto 0,80 que corresponde a un 80 %), un proceso ejecutado en segundo plano realiza el volcado del contenido del buffer a disco (Partitions). Mientras el volcado tiene lugar, la tarea *map* sigue escribiendo en el buffer mientras éste tenga espacio libre. El volcado se realiza siguiendo una política round-robin en un subdirectorío específico creado para el trabajo en ejecución. Este subdirectorío reside bajo el directorío especificado en la propiedad *mapred.local.dir*. Se crea un nuevo fichero cada vez que el buffer alcanza el porcentaje de ocupación fijado. A estos ficheros se les denomina ficheros de *spill*.

- **Split** (Particionamiento): antes de escribir a disco, un proceso en segundo plano divide la información en tantas particiones ($r_1, r_2, \dots, r_m$) como tareas *reduce* se han programado para el trabajo en curso.
- **Sorting**: se realiza en memoria una ordenación por clave de cada partición. En el caso que la aplicación hubiera definido una función *combiner*, ésta procesará la salida generada por la ordenación. El propósito de la función *combiner* es reducir el volumen de datos a entregar a la fase *reduce* agrupando los registros que comparten la misma clave.
- **Merge**: antes de que la tarea *map* finalice, los diferentes ficheros de *spill* generados son intercalados generándose un fichero de salida ordenado. La propiedad *io.sort.factor* controla al máximo número de ficheros a fusionar a la vez, que por defecto es 10.

Tarea *reduce*

Los ficheros particionados generados en la fase *map* son proporcionados a las tareas *reduce* (*reduce* 1,..., *reduce* m) vía HTTP. El número de hilos usados para servir dichos ficheros viene fijado por la propiedad *tasktracker.http.threads* y por defecto está fijado a 40 hilos para cada TaskTracker. Presenta las siguientes fases:

- **Copy**: Tan pronto como las tareas *map* finalizan la información correspondiente a cada tarea *reduce* es copiada. La fase *reduce* tiene un número de hilos que realizan dichas copias. Dicho número está fijado por la propiedad *mapred.reduce.parallel.copies* (por defecto 5). La información de salida que genera el *map* es copiada a un buffer del TaskTracker controlado por la propiedad *mapred.job.shuffle.input.buffer.percent* (por defecto 0,70 que corresponde al 70 %) que indica la proporción de memoria heap utilizada para tal propósito. Cuando dicho buffer llega al porcentaje de ocupación indicado por la propiedad *mapred.job.shuffle.merge.percent* (por defecto 0,66 o 66 %), o se ha llegado al número máximo de ficheros de salida de tareas *map* escritos en el búfer, fijado por la propiedad *mapred.job.shuffle.merge.percent* (por defecto 1.000), son fusionados y volcados a disco. Dado que las copias se acumulan en disco, un proceso en segundo plano los fusiona y ordena en ficheros más grandes ahorrando tiempo en subsiguientes fusiones.
- **Merge**: Cuando todos los ficheros de salida de las tareas *map* han sido copiados, son intercalados manteniendo la ordenación con la que se han generado en el proceso *map*. Este proceso se realiza en varias iteraciones, cuyo número viene dado por el resultado de dividir el número de ficheros *map* por la propiedad *io.sort.factor* (por defecto 10). Por ejemplo, si se generaron 40 ficheros en la fase *map*, se realizarán cuatro iteraciones. En la primera se intercalarán 10 ficheros en uno sólo y así sucesivamente. Al finalizar se habrán obtenido cuatro ficheros que se pasan directamente a la fase *reduce*.

- ***reduce***: Se invoca a la función *reduce* por cada clave extraída del fichero generado en la fase Merge. La salida de esta fase puede ser grabada directamente en el HDFS, o bien a través de una función auxiliar para grabar la información con un formato determinado. Se genera un fichero en el HDFS por tarea *reduce* ejecutada.

Bibliografía

- [EBT17] Allae Erraissi, Abdessamad Belangour, and Abderrahim Tragha. A comparative study of hadoop-based big data architectures. *Int. J. Web Appl.*, 9(4):129–137, 2017.
- [EN16] Boris Evelson and Native Hadoop BI Is A New. The forrester wave™: Native hadoop bi platforms, q3 2016. 2016.
- [Sar14] Debarchan Sarkar. *Pro Microsoft HDInsight: Hadoop on Windows*. Apress, 2014.